华为 SSD 读写冲突论文研读

Zhejiang University

薛森

2022 年 3 月 22 日

目录

Myoungsoo Jung 博士论文：DESIGN OF A SMART NON-VOLATILE MEMORY CONTROLLER: ARCHITECTURE MODELING, SYSTEMS ANALYSIS, PARALLEL I/O PROCESSING AND SCHEDULING ALGORITHMS

1. 第六章：Memory Request Scheduling
2. 第七章：Garbage Collection Scheduling

SSD 结构

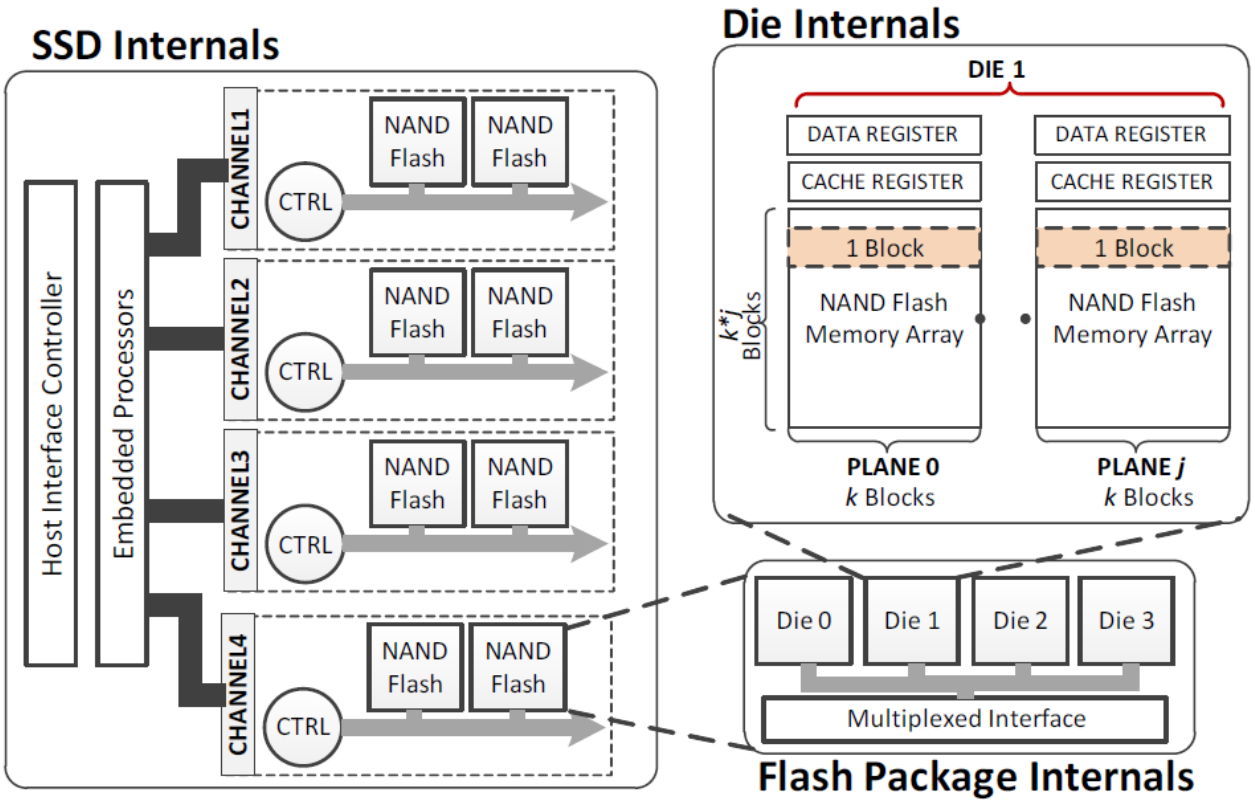


图: Physical internal architecture of SSD

SSD 软件结构

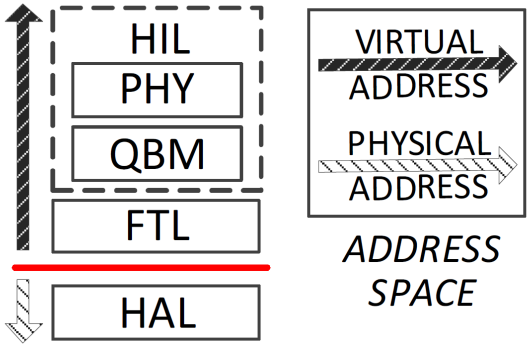


图: SSD Software Structure

- 主机接口层 (Host Interface Layer, HIL)：负责主机系统和 SSD 内底层之间的通信。原始通信协议部分由物理层（Physical layer, PHY）处理，而与 I/O 调度和缓冲相关的职责由队列和缓冲区管理层（Queue and Buffer Management layer, QBM）处理。
- 闪存转换层 (Flash Translation Layer, FTL)：FTL 负责主机地址空间和物理地址空间之间的地址转换。
- 硬件抽象层 (Hardware Abstraction Layer, HAL)：HAL 是一种设备驱动程序，用于管理物理 NAND 闪存。它负责将 NAND 指令提交

问题概述

目的：减少延迟

对于 NAND 闪存，闪存单元偏向于读取，读取性能通常是写入性能的 10 到 40 倍。

- 闪存中的三个基本操作（读取、写入和擦除）的持续时间差异很大。
- 读操作需要数十微秒；写操作需要数百微秒；擦除操作需要数千微秒。

如果要写入的块尚未空闲，则必须在写入之前进行擦除，这一要求加剧了这种差异。

- 如果对无效数据占用的块进行写入，则会产生擦除延迟加上写入延迟。
- 如果写入的块需要保留其中的现有数据，这些情况需要所有三种操作：首先执行读取，然后执行擦除，最后使用新旧数据混合写入块。这时 NAND 在时间上不占优势。

问题分析

由于数据布局的独立性，写操作的争用性较低，并行性较高，而读操作的并行性则不那么容易实现。具体地说，为了进行读取，对于要读取一些数据，并且这些数据必须驻留在特定的块中。这导致了对数据布局和访问顺序的严格性能依赖，导致请求队列大量涌现，从而扼杀了并行性

QBM 目前处于 FTL 之上的位置，对于这种性能依赖性无能为力，因为它只知道虚拟地址——无法智能地重新排序访问以减少冲突。

实验

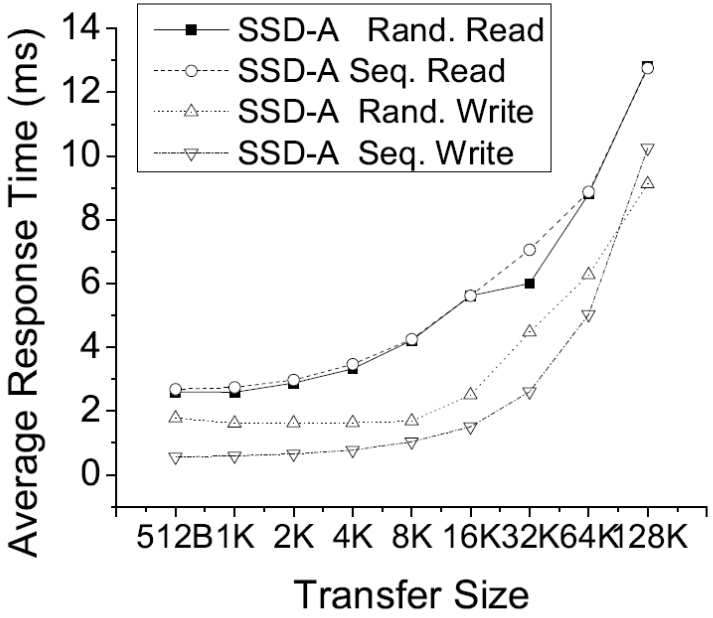
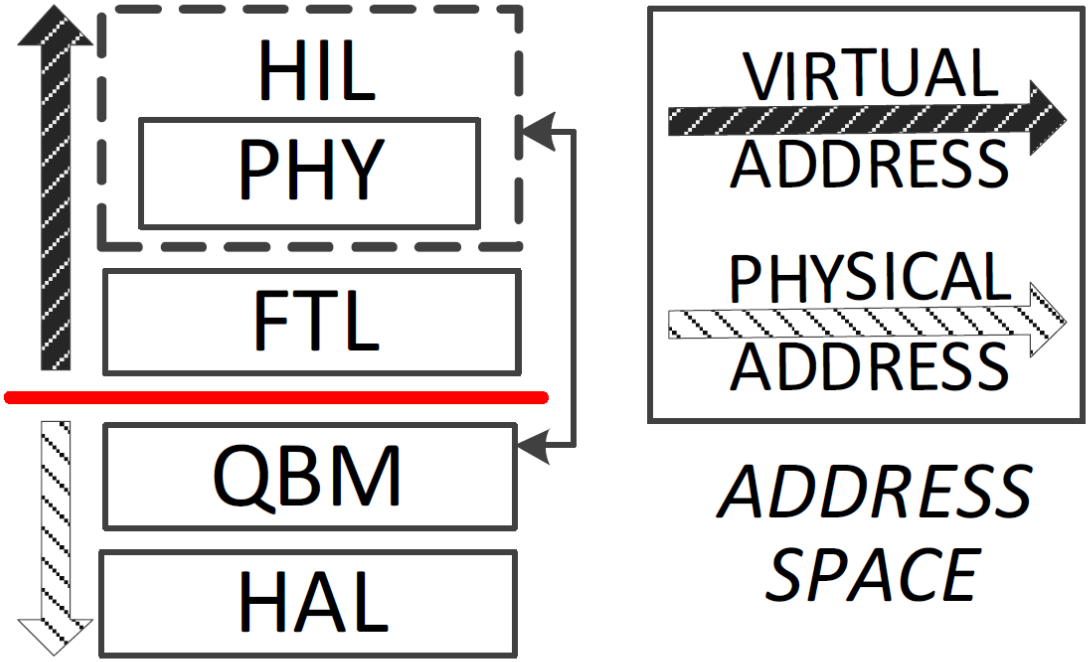


图: SSD Read/Write Experiment

在测试案例中，随机写入都优于随机读取.

解决方案



图：New SSD Software Structure

解决方案

物理地址排序 (Physical Addressed Queuing, PAQ)
冲突分类

- Domain: Channel 请求相同的指令集合
- Cluster: Flash Package 请求相同的指令集合
- Node: DIE 请求相同的指令集合

可以看出，上一级的冲突肯定包含了下一级的冲突。

启发式规则

原则

- 低级别的冲突代价最高。
- 可以在执行其他工作时从冲突集合调度单个指令，其后再执行其他有冲突的操作，这样可以在实现并行性的同时避免争用。

启发式规则

1. 首先添加在最低级别发生冲突的指令。
2. 避免指令集包含节点级和集群级的冲突。
3. 向指令集中添加指令，优先添加低级冲突，直到无法在不违反 (2) 的情况下添加更多指令。

思路

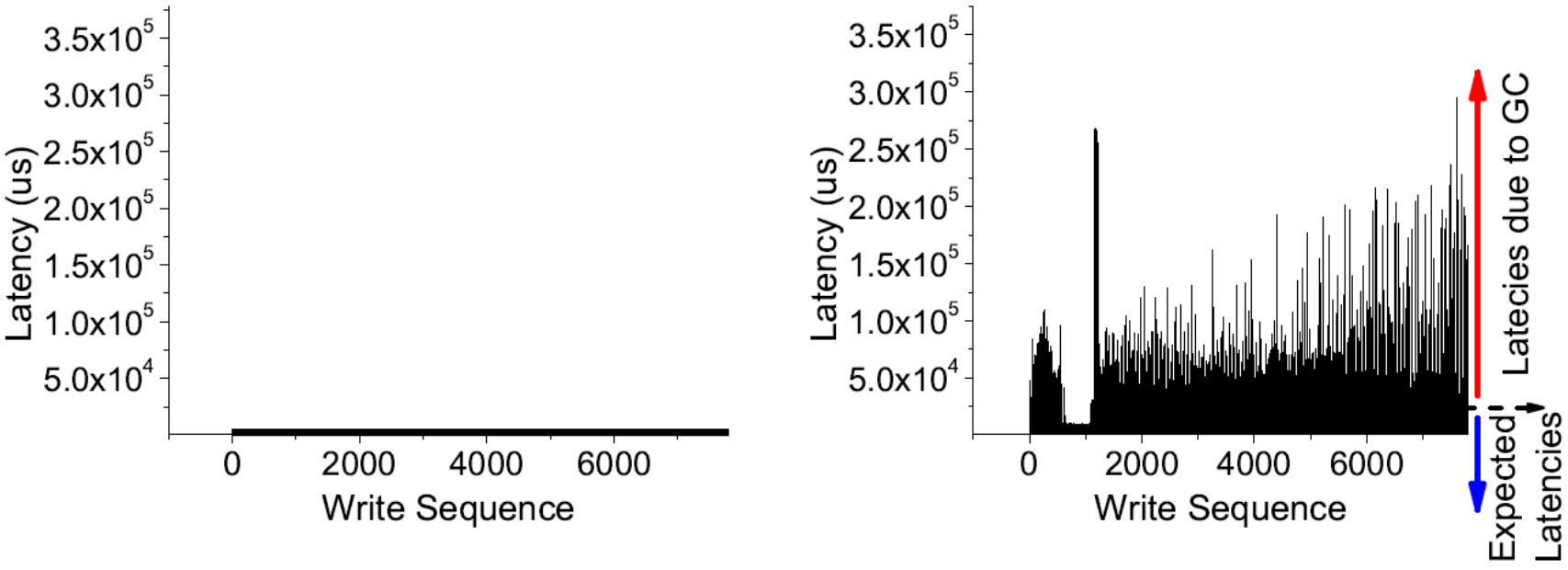
1. 能否证明是最优的，优化空间？
2. 考虑 GC 建模

GC 问题概述

动机：减少延迟，增加带宽

- 对于 I/O 队列，闪存转换层 (Flash Translation Layer, FTL) 需要将虚拟地址转化为物理地址（准备好一个物理块 (physical block)）。当物理块 (physical block) 没有足够的页 (page) 时，需要启动 GC(Garbage Collection) 操作。
- GC 操作将要更新的块和原始块擦除，将其中的有效数据转移到一个新的块中。所以 GC 比普通的 I/O 操作更耗时。

实验验证

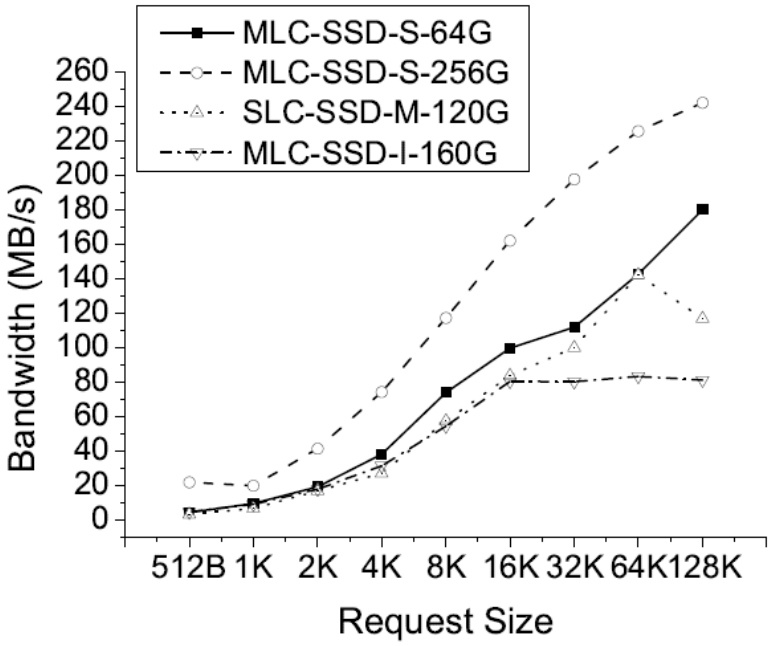


(a) Normal Latencies

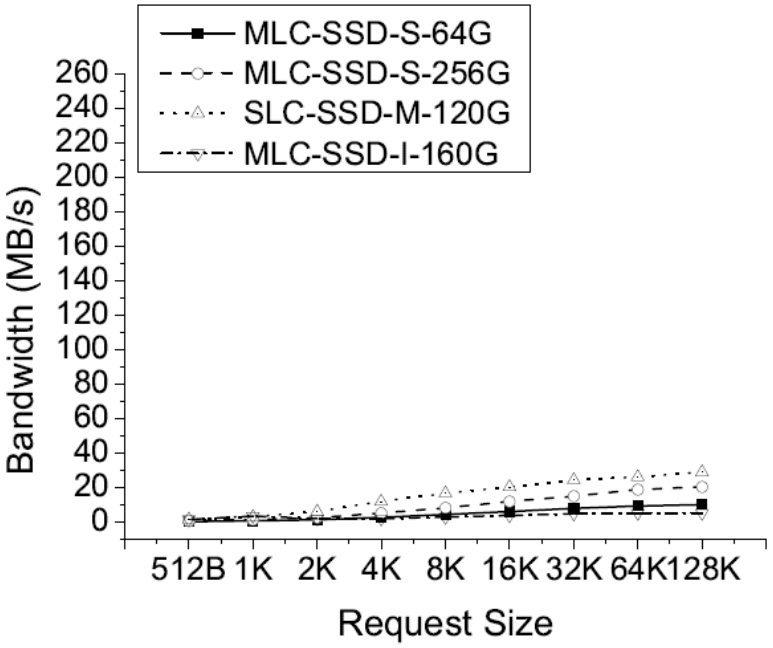
(b) Latencies with GCs

图: Latency Comparison

实验验证



(a) Normal Bandwidth (Pristine)



(b) Bandwidth with GCs (Fragmented)

图: Bandwidth Comparison

空闲时间

为了避免 GC 造成的功能降低，可以考虑将 GC 转移到设备空闲时间进行。

- 短空闲时间 (short idle periods)。有些应用产生的 I/O 序列当中会有短时间间隔。实验当中约 $1.8\mu s$ 至 $15.2ms$。可以在 FTL 层通过以下公式预测 I/O 命令的时间间隔。

$$T_{\text{short-idle}} = t_{\text{start}_{i+1}} - (t_{\text{start}_i} + t_{\text{exe}_i} * l_i)$$

- 长空闲时间 (long idle periods)。有些应用会有较长的 (1s) 的时间间隔，其间没有 I/O 指令发出。

GC 策略

1. 提前 GC(Advanced GC, AGC)
   * 短空闲时间：前瞻垃圾回收 (look-ahead garbage collection)
   * 长空闲时间：主动块压缩 (proactive block compaction)
2. 延迟 GC(Delayed GC, DGC)
   - 替换块更新 (Update Block Replacement)
   - 追溯块压缩 (Retroactive Block Compaction)

提前 GC

- 前瞻 GC(look-ahead garbage collection)
  1. 计算可以在短空闲时间内执行的 GC 操作的数量
  2. 计算 GC 延迟
  3. 提前在短空闲时间按需执行 GC 操作
- 主动块压缩 (proactive block compaction)
  检测逻辑块中被有效/无效页面完全占用的块，并在长空闲期间提前压缩它们。将完全占用的物理块中的所有有效页转移到新的干净块，并擦除所有无效页。

延后 GC

- 替换块更新 (Update Block Replacement)
  将页面迁移活动推迟到将来的空闲时间
- 追溯块压缩 (Retroactive Block Compaction)
  查询逻辑块对应的块的物理块地址 (Physical Block Numbers)，压缩这些块并将它们返回到空白状态。

实验结果

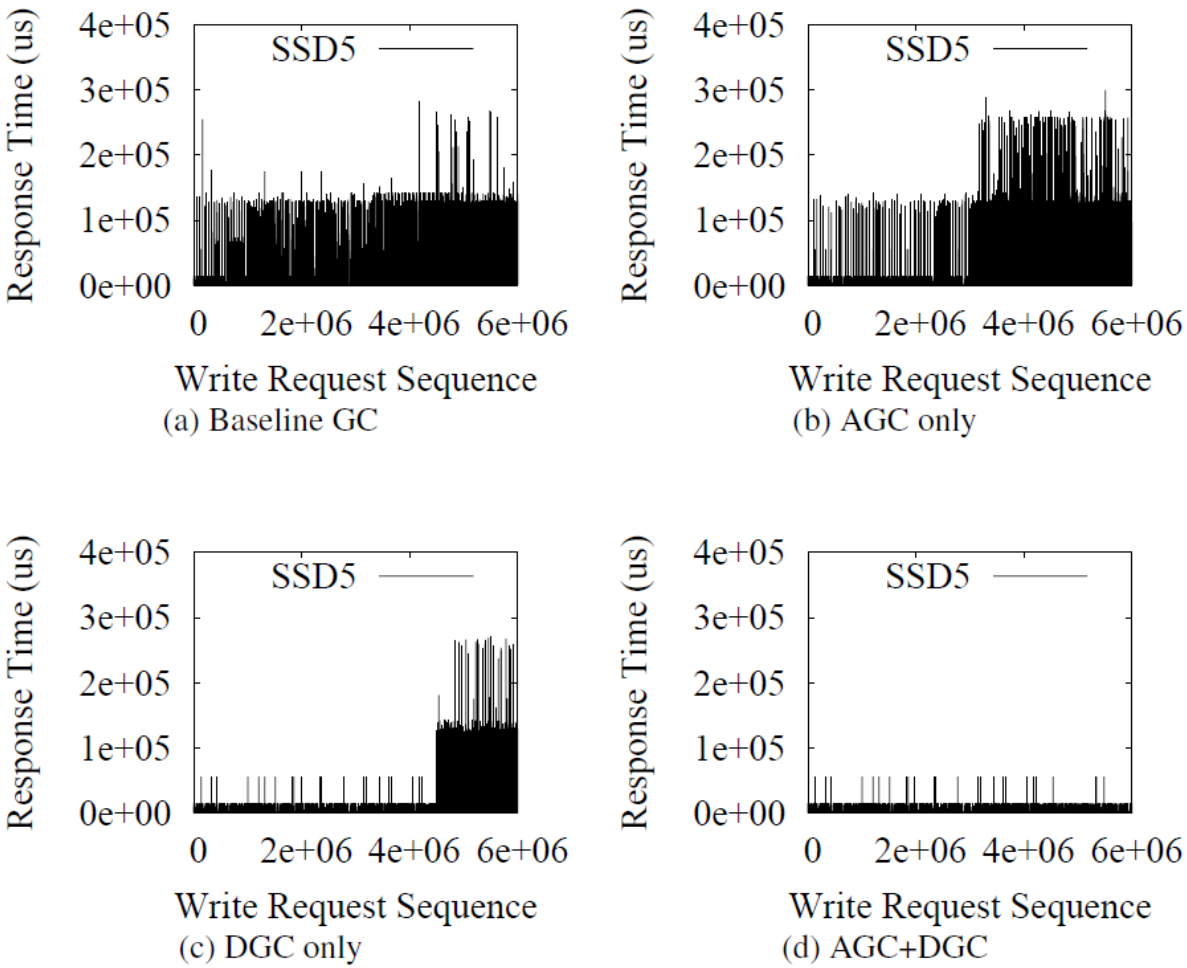


图: AGC and DGC performance

思路

1. GC 的优化空间
2. 通过 workload 特性建模